

Core_{sk}

ANONYMOUS AUTHOR(S)

In recent years, significant advances have been made in numerical debugging and static analysis tools. Recent debugging tools significantly reduce overhead using double-double oracles, while recent static analysis tools accurately bound rounding errors using condition numbers. However, these new techniques also have downsides: double-double oracles can miss floating-point errors due to correlated errors, while condition number formalisms cannot cleanly handle over- and underflow. We show that combining both techniques avoids the downsides of each and implement this combination in EASTEREGG. EASTEREGG uses double-double computations but not as an oracle; instead, it detects erroneous operations using condition numbers, which minimizes overhead while achieving precision and recall similar to more expensive methods. EASTEREGG also introduces a separate oracle to detect harmful over- and underflows more accurately; we implement this oracle in a new performant number representation. As a result, EASTEREGG achieves a precision of 80.0% and a recall of 96.1% on a collection of 500 difficult numeric benchmarks. An ablation study shows that EASTEREGG combines the strengths of both double-double oracles and condition numbers: it is more accurate than double-double oracles yet dramatically faster than methods based on arbitrary-precision condition number computations.

ACM Reference Format:

Anonymous Author(s). 2026. Core_{sk}. 1, 1 (February 2026), 6 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 INTRODUCTION

2 SKIA

We define 2 languages in this section, Surface_{Skia} and Core_{Skia}. Surface_{Skia} corresponds to the SkCanvas API a user of Skia typically uses to draw pictures.

Surface _{Skia}	s	::=	c
Command	c	::=	Draw(g, p) DrawText(string, p) Clip(cl , Intersect Difference) Save(); c ; Restore() SaveLayer(p); c ; Restore() $c; c$
Geometry	g, cl	::=	Rect(...) RRect(...) Path(...)
Paint	p	::=	Paint(f, st, c_f, b)
Fill	f	::=	Color(pixel) LinearGradient(...) ...
Style	st	::=	Solid Stroke
Filter	f	::=	Id Luma ...
BlendMode	b	::=	SrcOver DstIn Src ...

A drawing described by the SkCanvas API, and by extension a Surface_{Skia} program, is managed by a canvas. A canvas maintains 2 types of state: a layer state and a clip state. The layer state describes the pixel buffer currently managed by the program. It can also be described as the current

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

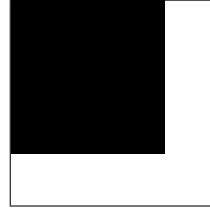
© 2026 Association for Computing Machinery.

XXXX-XXXX/2026/2-ART \$15.00

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

surface that is being drawn into. While a layer is conceptually infinite, only the part of the layer that is within the set bounds of the canvas. is “drawn”. Consider the following `SurfaceSkia` program, which draws into a canvas of size 256×256.

```
Draw(Rect(0, 0, 192, 192), DEFAULT_PAINT)
```

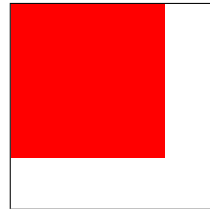


The layer state before the `Draw` command is an empty pixel buffer. The layer state after the `Draw` command is a pixel buffer with a black square. It’s easy to see how a `Draw` command manipulates the layer state by the drawing a specified geometry onto the layer. But just “drawing” geometries on a layer is a pretty bad drawing API as we have no way to describe how the pixels within the geometry are filled. That role is filled by `Paints`.

`Paint(f, st, cf, b)`

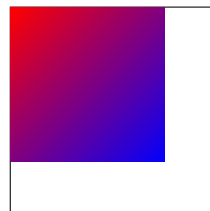
A `Paint` in `SurfaceSkia` is made of 4 components: fill, style, color filter and blend mode. Each of these components change how a geometry is drawn. The fill defines the mapping from geometry to pixel color values in the buffer.

```
Draw(Rect(0, 0, 192, 192),
    Paint(Color((1, 1, 0, 0)), ...))
```



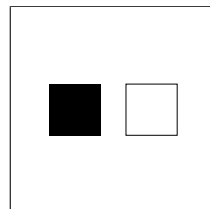
A fill does not need to be a constant fill of the same pixel. More interesting fills can be implemented with gradients:

```
Draw(Rect(0, 0, 192, 192),
    Paint(LinearGradient(...), ...))
```



A style modifies the geometry that is being drawn. The `Solid` style, the default style, fills the entire geometry. While a `Stroke` style only fills the boundaries of the geometry.

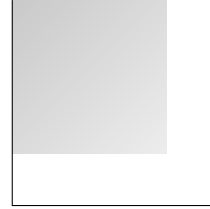
```
Draw(Rect(48, 96, 112, 160),
    Paint(..., Fill, ...));
Draw(Rect(114, 96, 208, 160),
    Paint(..., Stroke, ...))
```



A color filter further modifies the pixels filled in by `Fill`. It can be considered as a function from pixels to pixels. The default color filter is basically a function that returns pixels unmodified. The following example applies a “luminance filter” over the earlier gradient example. It turns

brightness, as perceived by the human eyes, into transparency. Brighter pixels are more opaque, and darker pixels more transparent.¹

```
Draw(Rect(0, 0, 192, 192),
      Paint(..., LumaFilter, ...))
```



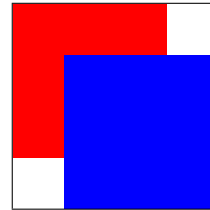
Finally, a blend mode describes how the geometry is drawn on to existing pixels of the current layer state. A blend mode is essentially a binary operator over pixels. The default blend mode is `SrcOver`, and it describes what is commonly known as “alpha compositing”. We may even write down `SrcOver` as an algebraic equation over pixels.

$$\text{SrcOver}(d, s) = s + (1 - s.\text{alpha}) \times d$$

2

We follow the usual terminology used in describing blend modes: s or *source* are the pixels being drawn while d or *destination* are the pixels being drawn on. Under `SrcOver`, the source pixel is placed over the destination pixel with the destination contribution reduced according to the source’s opacity. When the source pixels are opaque, the results is effectively like drawing the source pixels “over” the destination pixels. In the following example, we blend one square over another using `SrcOver` blending.

```
Draw(Rect(0, 0, 192, 192),
      Paint(Color((1, 1, 0, 0)), ..., SrcOver));
Draw(Rect(64, 64, 256, 256),
      Paint(Color((1, 0, 0, 1)), ..., SrcOver))
```

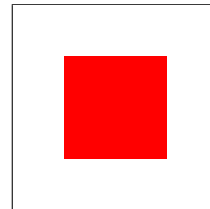


Another blend mode commonly used is `DstIn`.

$$\text{DstIn}(d, s) = d \times s.\text{alpha}$$

If the fill of the source geometry is opaque, the source acts like a “mask”. The pixels of `dst` that are not under the opaque pixels of source are masked out. The following program is the `SrcOver` example, but the second square has its blend mode changed to `DstIn`.

```
Draw(Rect(0, 0, 192, 192),
      Paint(Color((1, 1, 0, 0)), ..., SrcOver));
Draw(Rect(64, 64, 256, 256),
      Paint(Color((1, 0, 0, 1)), ..., DstIn))
```



While `DstIn` can be used to mask destination pixels in this way, `Surfaceskia` provides a dedicated clipping command for expressing this operation more directly.

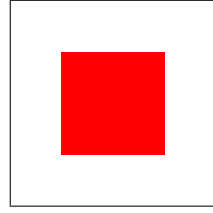
¹Red is brighter than blue if we consider their relative luminances.

²Instead of describing the blend mode separately over each channel of the pixel, we instead describe it over all channels simultaneously.

```

148
149 Clip(Rect(64, 64, 256, 256), Intersect);
150 Draw(Rect(0, 0, 192, 192),
151     Paint(Color((1, 1, 0, 0)), ..., SrcOver))
152
153
154

```

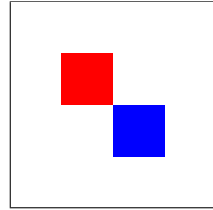


Now instead masking being a consequence of blending, we mask by constraining the draw geometry by intersecting it with a clip geometry. Clip commands modify the clip state for the current layer; A clip will affect all consequent Draw commands, as you can see with the following example.

```

159
160 Clip(Rect(64, 64, 192, 192), Intersect);
161 Draw(Rect(0, 0, 128, 128),
162     Paint(Color((1, 1, 0, 0)), ..., SrcOver));
163 Draw(Rect(128, 128, 256, 256),
164     Paint(Color((1, 0, 0, 1)), ..., SrcOver))
165
166

```

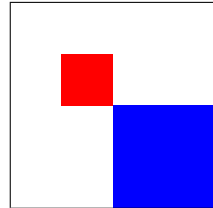


If we only use Clip and Draw commands, there is no way to reset the global clip state. In `SurfaceSkia` this is accomplished with the compound `Save(); ...; Restore()` commands and by using a *global clip stack* instead of a singular global clip state. A `Save()` duplicates the current top clip state on the clip stack. All subsequent Clip commands will modify the new top clip state. A `Restore()` will then pop the clip state duplicated by `Save`, discarding all changes made to the that clip state. To understand this better, let's surround both the Clip and the first Draw command of the previous example with `Save(); ...; Restore()`. The Clip will now only affects the first square and not the second one.

```

174 Save();
175     Clip(Rect(64, 64, 192, 192), Intersect);
176     Draw(Rect(0, 0, 128, 128),
177         Paint(Color((1, 1, 0, 0)), ..., SrcOver));
178 Restore();
179 Draw(Rect(128, 128, 256, 256),
180     Paint(Color((1, 0, 0, 1)), ..., SrcOver))
181

```

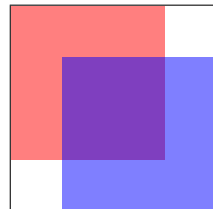


It is useful to understand `Save(); ...; Restore()` commands as a grouping operation; the effects of Clips within a `Save(); ...; Restore()` group are limited to only to the Draws within the group. But with the current set of commands, we do not have a way to “group” the consequences of layer state modifying commands. Consider the [SurfaceSkia program 6](#). What if we want to make the whole drawing translucent. As a first try we could decrease the opacity of each square.

```

187
188 Draw(Rect(0, 0, 192, 192),
189     Paint(Color((0.5, 1, 0, 0)), ..., SrcOver));
190 Draw(Rect(64, 64, 256, 256),
191     Paint(Color((0.5, 0, 0, 1)), ..., SrcOver))
192
193

```



This is not what we intended to draw with this program. `SaveLayer(p); ...; Restore()` commands help us group drawing effects in a useful way.

```

Image  i   : Point → Color
Shape  g   : Point → bool
Blend  b   : Color × Color → Color

```

Fig. 1. Our abstract model consists of Images and Shapes over abstract types Pixels and Colors, which can be blended together to form images.

```

CoreSkia  s ::= l
Layer     l ::= Empty()
           | Draw(l, g: Geometry, p: Paint)
           | SaveLayer(l, l: Paint)

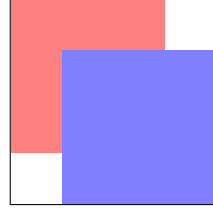
```

Fig. 2

```

SaveLayer(Paint(Color(0.5, 0, 0, 0)), ..., SrcOver);
  Draw(Rect(0, 0, 192, 192),
    Paint(Color((1, 1, 0, 0)), ..., SrcOver));
  Draw(Rect(64, 64, 256, 256),
    Paint(Color((1, 0, 0, 1)), ..., SrcOver));
Restore()

```



This is accomplished by extending the global layer state to a stack. All Draw operations occur on the current top layer state. A SaveLayer, like Save, duplicates the current clip on the stack. It also pushes a new layer on the layer stack. All operations after a SaveLayer now modify the new top layer on the stack. Just like Save, a matching Restore will pop the clip state stack. It will also pop the the top layer of the layer stack, and then “draw” it on layer below it. Just like Draw, Paint controls how the top layer is combined with the bottom layer. Like in the example above, Fill controls the overall opacity of the layer before combining. It is important to note that only the alpha channel of the fill is used, all other channels are useless to SaveLayer.³ Style is not used because SaveLayer has no geometry to draw. A Filter is applied to the entire Layer just like in Draw. Finally, BlendModes describe how the top layer is combined on the bottom layer.

3 SEMANTICS

Our semantics of Skia is built around a simple abstract model of images and shapes, shown in fig:model. - Abstract type of Pixels and Colors - Image is Pixel → Color - Shape is Pixel → Bool - Blend is Color → Color → Color Skia works by taking Shapes; filling them with primitive Images; Blending the results together in various orders

However, Skia does not permit arbitrary drawing and blending operations; instead, these always happen in a prescribed order. We thus introduce the Layer abstraction, shown in fig:layer, which defines the core Draw and BlendLayer operations in Skia. We focus on these two operations specifically because they are the core target of the optimizing compiler described in sec:optimize. The Layer constructors can be defined in terms of the abstract model, as shown in fig:denote. - Rename SaveLayer to BlendLayers(*l*, *l*, *p*) - Define our 3 constructs in terms of primitive operations

We can now move on to defining the grammar of Skia.

As our main goal is to try and verify rewrites in Skia, it is hard to do so over Surface_{Skia}. This is mainly because of the hidden layer and clip state of Surface_{Skia}. That is why we introduce a new

³If the Fill is a gradient, the alpha is assumed to be 1

language called $\text{Core}_{\text{Skia}}$, which is a declarative version of $\text{Surface}_{\text{Skia}}$. $\text{Core}_{\text{Skia}}$ makes layers the main value type of the language, and also make explicit the clip state at each draw command. This also has the consequence of completely removing Saves from $\text{Core}_{\text{Skia}}$.

We need to talk about how to convert $\text{Surface}_{\text{Skia}}$ to $\text{Core}_{\text{Skia}}$. Not sure how to do that.

To verify rewrites in $\text{Core}_{\text{Skia}}$ we define semantics for it. We define a denotation function $\llbracket \cdot \rrbracket$ over Layers.

$$\llbracket \text{Layer} \rrbracket : \text{Point} \rightarrow \text{Pixel}$$

That is, A Layer is a map from Points to Pixels, or more literally a buffer of pixels, indexed by points. We now define the denotation function over the 3 types of Layers. First, the Empty layer.

$$\llbracket \text{Empty}() \rrbracket = \lambda pt.(0, 0, 0, 0)$$

Next, Draw.

```

     $\llbracket \text{Draw}(l, g, p, c) \rrbracket =$ 
    let l_b := denote l_b
    let (fill, blend_mode, style, filter) := paint
    let drawPx := if (style geo) pt && (clip pt)
                  then ((getFillFunc fill) pt)
                  else Pixel.Transparent
    blend_mode (l_b pt) (filter drawPx)

```

idk how to typeset this man

Next, Draw.

```

     $\llbracket \text{SaveLayer}(l_b, l_t, g, p) \rrbracket =$ 
    let l_b := denote l_b
    let l_t := denote l_t
    let blend_mode : BlendMode := paint.2.1
    let filter      : Filter    := paint.2.2.2
    let alpha       : Real      := getAlpha paint.1
    blend_mode (l_b pt) (filter (applyAlpha alpha (l_t pt)))

```

Argument symbols are all over the place. There is applyAlpha here for som god forsaken reason. I am very sleepy.

4 COMPILER

5 EVALUATION

6 RELATED WORK

7 CONCLUSION